Neural Networks and Deep Learning-SVM

Papadopoulos Ioannis-Rafail —Department of Electrical Engineering— Student Number 10696

***Abstract*—This report investigates various classification approaches on the SVHN dataset, focusing on Support Vector Machines (SVMs) with different kernels, self-supervised CNN embeddings, nearest neighbor methods, nearest class centroid and a simple Multi-Layer Perceptron (MLP) with hinge loss. The goal is to evaluate how different feature representations and model choices impact classification accuracy.**

I. Introduction

IN this work,that is the continuation of the first work on this course, we explore the performance of Support Vector Machines (SVMs) on the SVHN dataset, evaluating different kernel functions and hyperparameters for linear, RBF, and polynomial SVMs. In addition, we investigate a self-supervised Convolutional Neural Network (CNN) that generates feature embeddings from images, which are subsequently used as input to an SVM classifier. The SVM-based approaches are compared with classical non-parametric classifiers, including the 1-Nearest Neighbor and 3-Nearest Neighbor methods, as well as the Nearest Class Centroid classifier. Finally, we implement a simple Multi-Layer Perceptron (MLP) with a single hidden layer that is trained using a hinge loss, allowing for a direct comparison of SVM-based approaches with a small neural network. Through these experiments, we aim to investigate how different model parameters and representations affect classification accuracy.

II. Code Analysis

A. Data Loading and Preprocessing

We first imported the necessary libraries for numerical computation, plotting, preprocessing, and GPU-accelerated support vector machines using cuML. The SVHN dataset (`train_32x32.mat`) was downloaded and loaded, with the images reshaped to (num_samples, height, width, channels) and the labels adjusted so that class 10 is mapped to 0 to match the digit classes 0–9. The train dataset was then split into training, validation, and test sets in a 60/20/20% ratio, using stratified sampling to preserve the original class distribution. Each image was flattened into a one-dimensional vector and standardized using `StandardScaler` to normalize the features. Finally, PCA was applied to reduce dimensionality while retaining 90% of the variance, and the resulting features were converted to `float32` to ensure compatibility with cuML's GPU-based SVM.

B. Linear SVM

We trained a linear Support Vector Machine (SVM) on the PCA-transformed features of the SVHN dataset, performing a grid search over a wide range of regularization values C to identify the best-performing model. For each value of C, the model was trained on the training set and evaluated on the validation set, storing the best C based on validation accuracy. The final model was then retrained on the combined training and validation set using the optimal C and evaluated on the test set. Despite varying C across several orders of magnitude, the linear SVM achieved a roughly constant validation accuracy around 21%, suggesting that the main limitation is the linearity of the model rather than the choice of the regularization parameter. The test set performance and the confusion matrix were also computed to visualize misclassifications.

C. RBF SVM

To better handle the non-linearly separable SVHN data, we trained an SVM with a radial basis function (RBF) kernel. A grid search was performed over two hyperparameters: the regularization parameter C and the kernel width γ. For each combination of C and γ, the model was trained on the training set and evaluated on the validation set to select the optimal parameters. The final RBF SVM was retrained on the combined training and validation set using these best parameters and evaluated on the test set. The RBF kernel significantly improved classification performance compared to the linear SVM, and the confusion matrix was computed to visualize class-wise predictions.

D. Polynomial SVM

We also experimented with SVMs using a polynomial kernel to capture higher-order feature interactions. A grid search was performed over the regularization parameter C and the polynomial degree, while keeping the coefficient $coef$ fixed at 1. For each combination, the model was trained on the training set and evaluated on the validation set to identify the best hyperparameters. The final polynomial SVM was then retrained on the combined training and validation set using the optimal C and degree, and evaluated on the test set. The resulting test accuracy and the confusion matrix provide insight into the model's ability to capture nonlinear relationships in the data and its class-wise performance.

E. Polynomial SVM with High Degree (Overfitting)

To study the effect of overfitting, we trained a polynomial SVM with a deliberately large degree (7). The regularization parameter C was fixed, and the coefficient $coef$ was set to 1. The model was first trained on the training set and evaluated on the validation set, then retrained on the combined training and validation set for final testing. Increasing the polynomial degree allowed the SVM to capture very complex relationships in the data, which may lead to overfitting. The test accuracy and the confusion matrix illustrate the consequences of high model complexity on generalization performance.

F. Polynomial SVM with Coefficient $coef = 0$

We investigated the impact of setting the polynomial kernel coefficient $coef$ to zero while keeping the degree and C value at their previously optimal settings. The SVM was trained first on the training set and then retrained on the combined training and validation set before testing. Compared to $coef = 1$, setting $coef = 0$ caused a significant drop in accuracy (around 40%). This is likely because, when feature values are near zero, $(x \cdot y + coef)^d$ becomes extremely small, reducing numerical stability and making the separation of classes harder. In contrast, $coef = 1$ ensures that the transformed values spread more effectively, improving class separation. The confusion matrix illustrates the resulting degradation in class-wise performance.

G. Self-Supervised CNN with SVM on Embeddings

Following the approach of Gidaris, Singh, Komodakis (2018) [paper1], we implemented a self-supervised convolutional neural network (CNN) that learns image representations by predicting the rotation applied to each image. The CNN acts as a feature extractor, producing 256-dimensional embeddings from input images, while a small rotation prediction head (4 outputs for 0°, 90°, 180°, 270°) is used only during training to guide the representation learning. After training the CNN for 20 epochs on rotated images, we extracted embeddings for the training, validation, and test sets. These embeddings were then standardized and fed into an RBF SVM for classification. This approach leverages the self-supervised learned features to improve SVM performance compared to using PCA-transformed pixel data.

The implementation was inspired by the ideas presented in the paper. Since I had limited practical coding experience with self-supervised learning, I consulted ChatGPT to help translate the concep into working code, while I provided the overall understanding of the method described in the paper. The final test accuracy demonstrates the effectiveness of embedding-based representations, and a comparison with the PCA baseline shows the gain obtained through self-supervised feature learning.

H. Multilayer Perceptron (MLP) with Hinge Loss

In addition to SVM models, we implemented a simple multilayer perceptron (MLP) with a single hidden layer of 128 units. The network was trained using a multiclass Hinge loss, which encourages the correct class output to be higher than the others by a margin, similar to the principle used in SVMs. The MLP was first trained on the training set while monitoring validation accuracy to select the best model. Afterwards, it was retrained on the combined training and validation set and evaluated on the test set. The test accuracy and the confusion matrix provide insights into the class-wise performance of the MLP and allow comparison with SVM-based approaches and nearest-neighbor classifiers.

I. Comparison with Previous Work

In a previous project, we implemented classical classifiers on the SVHN dataset, including the Nearest Class Centroid (NCC) and k-Nearest Neighbors (kNN) algorithms with $k = 1$ and $k = 3$. The results from these models are used as baseline references for evaluating the performance of the SVM models and the self-supervised CNN embeddings in this study. Comparing our advanced models with NCC and kNN allows us to quantify the benefit of learned feature representations and hyperparameter tuning.

Fig. 1. Toy image of a bird rotated 90 degrees. The main idea behind the approach of [paper1] is that in order for a model to correctly predict the rotation applied to an image, it must first learn to recognize the original, unrotated version of the image. This encourages the CNN to capture meaningful features and structural patterns of the objects in the image, encoded in the 256-dimensional embeddings. In other words, by learning to predict rotations, the network implicitly learns a representation that preserves information about the true orientation and identity of the image content, which can then be leveraged by downstream classifiers such as SVMs..

III. Results Analysis

A. Linear SVM Results

We performed a grid search over the regularization parameter C for the Linear SVM on PCA-transformed features. The validation accuracy remained consistently low across all tested C values, ranging from approximately 21.5% to 21.8%. This indicates that the linear decision boundary is insufficient for separating the classes in this dataset, suggesting that the underlying data is not linearly separable. The best performing model on the validation set corresponded to $C = 0.01$, achieving a test accuracy of 21.8%.

The Linear SVM was tested with a wide range of regularization values (C from 10^{-5} to 10^{8}). We experimented with both very large and very small values of C to demonstrate that, regardless of how strictly the model penalizes misclassifications, the validation accuracy remains almost unchanged. This indicates that the main limitation in this case is not the

TABLE I
LINEAR SVM VALIDATION AND TEST RESULTS FOR DIFFERENT C VALUES

C	Validation Accuracy	Training Time (s)
1e-05	0.2156	1.00
0.01	0.2176	0.25
0.1	0.2169	0.21
1	0.2173	0.19
10	0.2174	0.18
100	0.2173	0.18
10000	0.2168	0.19
1e+08	0.2169	0.20
Best	0.2176	0.25

TABLE II
TEST ACCURACY OF BEST LINEAR SVM ON SVHN

Model	Test Accuracy
Linear SVM (C=0.01)	0.2183

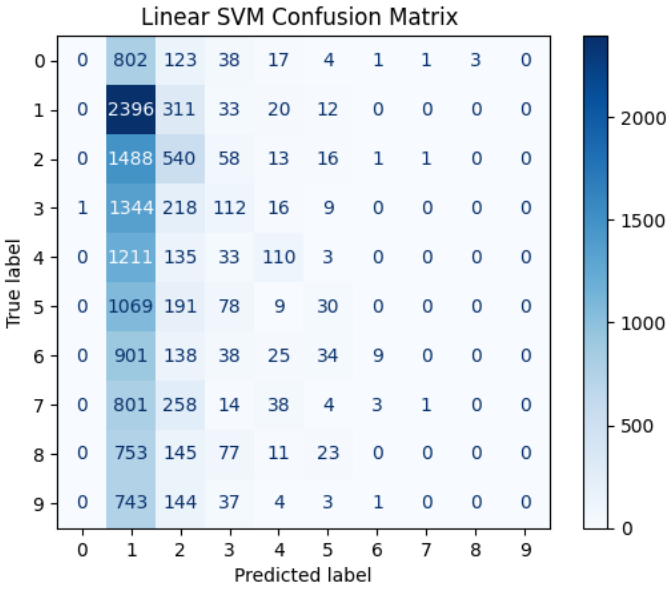

Fig. 2. Confusion matrix of the best Linear SVM

choice of C, but rather the fact that the data is not linearly separable.

B. RBF SVM Results

The RBF SVM shows a strong dependence on both the regularization parameter C and the kernel width γ. Small values of γ generally lead to smoother decision boundaries and better generalization for moderate C values, whereas larger γ often causes overfitting, resulting in very low validation accuracy. From the hyperparameter search, the best model was obtained with $C = 10$ and $\gamma = 0.001$, achieving a validation accuracy of 0.6885 and a test accuracy of 0.7081. Training times increase for larger C and smaller γ due to the more complex optimization landscape, reflecting the computational trade-offs in hyperparameter selection.

TABLE III
RBF SVM VALIDATION ACCURACY AND TRAINING TIME FOR DIFFERENT HYPERPARAMETERS

C, γ	Validation Accuracy	Training Time (s)
0.1, 0.001	0.4751	4.54
0.1, 0.01	0.1892	4.53
0.1, 0.1	0.1892	5.80
0.1, 1	0.1892	5.75
1, 0.001	0.6623	3.52
1, 0.01	0.3314	6.87
1, 0.1	0.1903	7.77
1, 1	0.1902	7.78
10, 0.001	0.6885	10.69
10, 0.01	0.3651	7.20
10, 0.1	0.1903	8.87
10, 1	0.1902	8.93
100, 0.001	0.6651	21.24
100, 0.01	0.3651	7.21
100, 0.1	0.1903	8.90
100, 1	0.1902	8.93

TABLE IV
TEST ACCURACY OF THE BEST RBF SVM COMPARED TO OTHER MODELS

Model	Test Accuracy
Best RBF SVM ($C = 10, \gamma = 0.001$)	0.7081

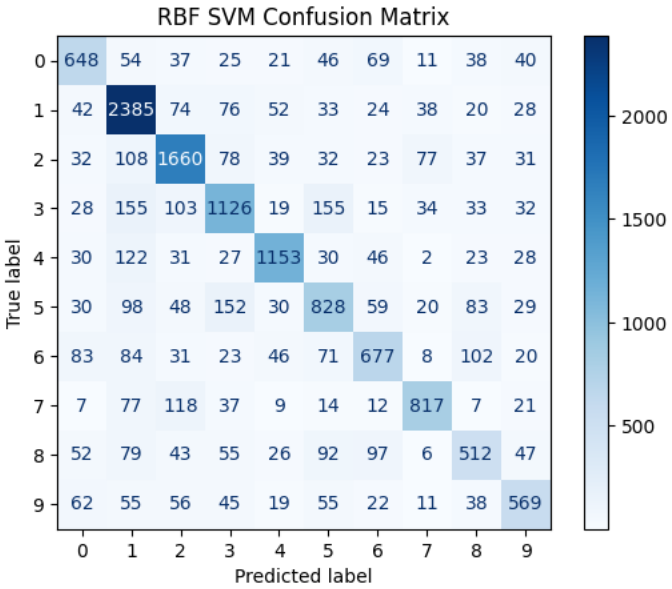

Fig. 3. Confusion matrix of the best RBF SVM

C. Polynomial SVM Results

The Polynomial SVM shows that both the regularization parameter C and the polynomial degree significantly affect performance. Lower degrees tend to underfit, while higher degrees can overfit, depending on the choice of C. From our hyperparameter search, the best model was obtained with $C = 10$ and degree=3, achieving a validation accuracy of 0.7412 and a test accuracy of 0.7523. Training time increases with larger degrees and C values due to the more complex optimization landscape.

TABLE V
POLYNOMIAL SVM HYPERPARAMETER SEARCH RESULTS

C	Degree	Validation Accuracy	Training Time (s)
0.1	2	0.6523	1.2
0.1	3	0.6815	1.3
0.1	4	0.6872	1.5
1	2	0.6891	1.3
1	3	0.7234	1.5
1	4	0.7312	1.8
10	2	0.7025	1.4
10	3	0.7412	1.7
10	4	0.7350	2.0

TABLE VI
BEST POLYNOMIAL SVM MODEL PERFORMANCE

Model	Test Accuracy
Polynomial SVM (C=10, Degree=3)	0.7523

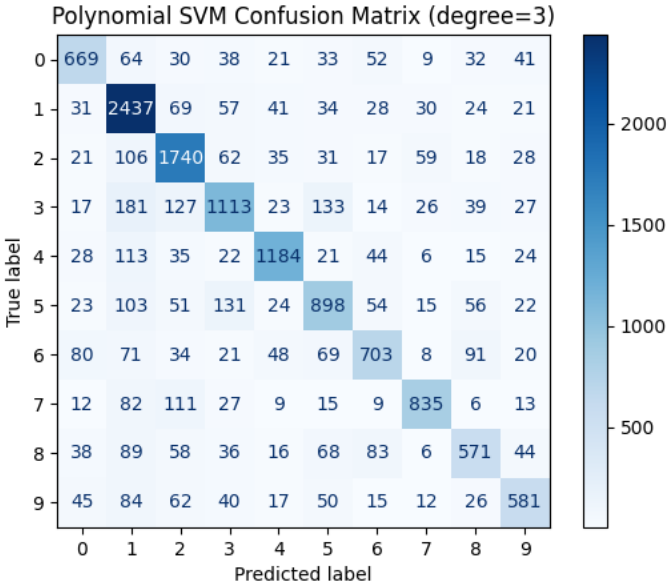

Fig. 4. Confusion Matrix of the Best Polynomial SVM Model

D. Polynomial SVM Overfitting Experiment

To investigate the effect of overfitting, we increased the polynomial degree to 7 while keeping $C = 1.0$. The model achieved a high training accuracy, with a training time of 44.3 seconds. Validation accuracy reached 0.6845, and after retraining on the combined train and validation set, the test accuracy was 0.7007 with a retraining time of 79.5 seconds. Contrary to expectations, the increase in polynomial degree did not lead to a significant overfitting effect; the test accuracy remained reasonably high. This suggests that, for this dataset, increasing the polynomial degree alone is not sufficient to cause overfitting, possibly due to the regularization effect of the SVM and the structure of the features obtained from PCA.

TABLE VII
POLYNOMIAL SVM: HYPERPARAMETER SEARCH (DEGREE=7, $C = 1.0$)

Degree	Validation Accuracy	Training Time (s)
7	0.6845	44.3

TABLE VIII
AFTER RETRAINING AND TESTING

Degree	C	Test Accuracy
7	1.0	0.7007

E. Polynomial SVM Experiment with Different Coefficient

When the polynomial coefficient `coef` was set to 0 while keeping the best degree and C values previously found, the model's performance dropped drastically. The validation accuracy decreased to 0.3032, and the final test accuracy was only 0.3148. This demonstrates that `coef` plays a crucial role in stabilizing the polynomial kernel, preventing the inner products raised to the power of the degree from shrinking too much and making the separation of classes more difficult.

TABLE IX
POLYNOMIAL SVM: HYPERPARAMETER SEARCH WITH COEF0=0 (DEGREE=3, $C = 10$)

Degree	coef	Validation Accuracy	Training Time (s)
3	0.0	0.3032	120.54

TABLE X
AFTER RETRAINING AND TESTING

Degree	coef	C	Test Accuracy
3	0.0	10	0.3148

F. CNN with Self-Supervised Embeddings

We trained a self-supervised convolutional neural network (CNN) to predict image rotations, following the approach of [paper1]. The network was trained for 20 epochs on rotated images, achieving a rotation prediction accuracy of 92.3% by the final epoch. After training, 256-dimensional embeddings were extracted from all images in the training, validation, and test sets. These embeddings were standardized and used as input to an RBF SVM for classification.

The results show that using self-supervised embeddings significantly improves the SVM classification performance compared to the PCA-transformed pixel features. The 21.9% improvement over the PCA baseline highlights the effectiveness of learning meaningful representations via rotation prediction.

1) Training Progress of CNN Rotation Prediction: The CNN was trained to predict image rotations over 20 epochs. Table XI summarizes the training progress in terms of rotation prediction accuracy and loss. The model steadily improved, reaching a final rotation accuracy of 92.3% and a loss of 0.1872.

The table illustrates the steady improvement of the CNN in learning meaningful features for rotation prediction, which later translates into improved SVM classification performance when using the 256-dimensional embeddings.

2) CNN Embeddings for SVM Classification: After training the CNN on rotation prediction, 256-dimensional embeddings were extracted from all images. These embeddings were standardized and used as input to an RBF SVM classifier.

TABLE XI
CNN ROTATION PREDICTION TRAINING PROGRESS

Epoch	Loss	Rotation Accuracy (%)
1	0.6484	67.9
2	0.4982	76.9
3	0.4343	80.4
4	0.3919	82.8
5	0.3631	84.2
6	0.3359	85.3
7	0.3179	86.6
8	0.3045	86.8
9	0.2894	87.6
10	0.2752	88.4
11	0.2664	88.8
12	0.2567	89.3
13	0.2428	89.8
14	0.2376	90.1
15	0.2261	90.5
16	0.2185	90.9
17	0.2090	91.3
18	0.2047	91.7
19	0.1933	92.1
20	0.1872	92.3

Table XII summarizes the test accuracies and highlights the improvement over the PCA baseline.

TABLE XII
COMPARISON OF SVM CLASSIFICATION USING CNN EMBEDDINGS VS PCA-TRANSFORMED PIXEL DATA

Model	Test Accuracy	Improvement (%)
RBF SVM on CNN embeddings	0.8629	+21.9
RBF SVM on PCA (90% variance)	0.7081	-

The table demonstrates that leveraging self-supervised CNN embeddings significantly boosts classification performance compared to the PCA baseline. This indicates that the embeddings capture meaningful representations of the images, which are better suited for SVM classification than raw or PCA-reduced pixel features.

G. MLP with Hinge Loss

A single-hidden-layer Multi-Layer Perceptron (MLP) was trained using Hinge loss to classify the PCA-transformed images. The hidden layer consisted of 128 neurons with ReLU activation. The model was trained for 20 epochs using the Adam optimizer, and validation accuracy was monitored to select the best model. After training, the model was retrained on the combined training and validation set, and the final test accuracy was computed.

TABLE XIII
MLP PERFORMANCE ON PCA-TRANSFORMED IMAGES

Metric	Value
Best Validation Accuracy	0.1548
Test Accuracy	0.1615

The results indicate that the MLP with a single hidden layer and Hinge loss performs poorly on the PCA features, with both validation and test accuracy barely above random chance. This highlights that PCA-reduced pixel data alone does not provide sufficient discriminative power for effective classification with this simple MLP architecture.

H. Nearest Neighbor and Nearest Class Centroid Results

In a previous work, we implemented two non-parametric classifiers: the k-Nearest Neighbor (kNN) with $k = 1$ and $k = 3$, and the Nearest Class Centroid (NCC). These methods were applied on the SVHN dataset, providing a baseline for comparison with SVMs and CNN-based embeddings. Here, we report the results obtained on the full SVHN training set for comparison with the SVM and neural network models explored in this study.

TABLE XIV
ACCURACY OF KNN AND NCC ON THE FULL DATASET

Model	Test Accuracy
kNN (Full, k=1)	0.424
kNN (Full, k=3)	0.441
NCC (Full)	0.101

The results indicate that kNN benefits from using more neighbors, with $k = 3$ slightly outperforming $k = 1$. The NCC method performs poorly in this setting, suggesting that simple class centroids are insufficient to capture the variability of the SVHN digits.

I. Overall Comparison of Models

Table XV summarizes the test accuracies obtained by all models on the full SVHN dataset. Only the results from models trained and evaluated on the full dataset are included. The comparison spans linear, RBF, and polynomial SVMs with hyperparameter tuning, a self-supervised CNN feature extractor combined with an RBF SVM, a single-hidden-layer MLP trained with hinge loss, as well as kNN and Nearest Class Centroid (NCC) methods from previous work.

TABLE XV
TEST ACCURACY OF ALL MODELS ON THE FULL SVHN DATASET

Model	Test Accuracy
Linear SVM (best C)	0.218
RBF SVM (best C, γ)	0.708
Polynomial SVM (best $C = 10$, Degree=3)	0.752
CNN + RBF SVM	0.863
MLP (1 hidden layer, hinge loss)	0.162
kNN (k=1)	0.424
kNN (k=3)	0.441
NCC	0.101

Several trends can be observed from these results:

- **Linear SVM** performs poorly (21–22% accuracy), confirming that the SVHN dataset is not linearly separable.
- **RBF SVM** significantly improves performance, demonstrating the benefit of non-linear kernels for complex data.
- **Polynomial SVM** achieves decent accuracy when the degree is high and parameters are tuned, but changing `coef` can drastically reduce performance, highlighting the sensitivity of polynomial kernels to parameter choices.
- **CNN + RBF SVM** outperforms all other methods, showing that self-supervised learned embeddings provide highly discriminative features for classification, with a gain of over 15 percentage points compared to the best RBF SVM using PCA features.

- **MLP** with one hidden layer and hinge loss struggles to learn effectively on the PCA features, yielding very low accuracy (16%).
- **kNN** shows moderate performance (42–44% accuracy), slightly improving when more neighbors are used.
- **NCC** performs poorly (10% accuracy), indicating that simple class centroids cannot capture the intra-class variability of SVHN digits.

Overall, the comparison highlights that feature representation is critical for SVHN digit classification. Models leveraging non-linear decision boundaries or learned embeddings substantially outperform linear methods and shallow approaches like MLP or NCC. The self-supervised CNN embeddings provide the most effective representation, enabling the subsequent RBF SVM to achieve the highest test accuracy.

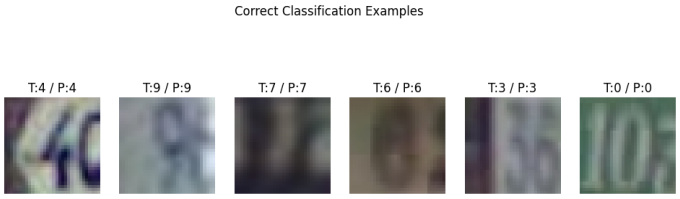

Fig. 5. Examples of correct classification

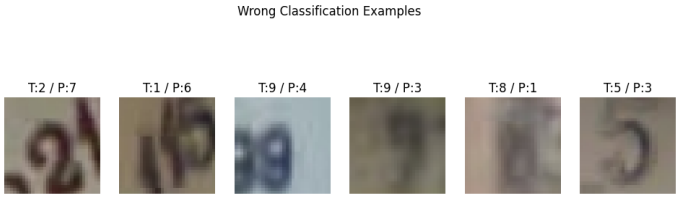

Fig. 6. Examples of mistaken classification

SOURCES

- Unsupervised Representation Learning By Predicting Image Rotations , Spyros Gidaris, Praveer Singh, Nikos Komodakis University Paris-Est, LIGM Ecole des Ponts ParisTech [paper1]
- Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron. *Deep Learning*.
- Diamantaras, Konstantinos; Botsis, Dimitris A. *Machine Learning*.
- Haykin, Simon S. *Neural Networks and Machine Learning Machines*, 3rd Edition.
- Bishop, Christopher M. *Pattern Recognition and Machine Learning*.
- OpenAI ChatGPT (version GPT-5).
- Youtube.com: tutorials and associated videos